Codio Project File Edit Find View Tools Education Help Run Virtual Desktop Debug

SANDIPAN-DEY

app.py × tonightwaiter-s...

```
    self.window.add_widget(self.add_button)
    self.window.add_widget(self.count_label)
    self.window.add_widget(self.subtract_button)

    self.add_button.bind(on_press=self.add_callback)
    self.subtract_button.bind(on_press=self.subtract_callback)

    return self.window

  def add_callback(self, instance):
    self.count += 1
    self.count_label.text = str(self.count)

  def subtract_callback(self, instance):
    self.count -= 1
    self.count_label.text = str(self.count)

if __name__ == "__main__":
  ClickButtons().run()
```

100% (57:23)

Terminal ×

```
[INFO   ] [Text        ] Provider: sdl2
[ERROR  ] [Input       ] MTDev is not supported by your version of linux
Traceback (most recent call last):
  File "/home/codio/anaconda3/envs/gui/lib/python3.10/site-packages/kivy/input/providers/__init__.py", line 41, in <module>
    import kivy.input.providers.mtdev
  File "/home/codio/anaconda3/envs/gui/lib/python3.10/site-packages/kivy/input/providers/mtdev.py", line 94, in <module>
    from kivy.lib.mtdev import Device, \
  File "/home/codio/anaconda3/envs/gui/lib/python3.10/site-packages/kivy/lib/mtdev.py", line 30, in <module>
    libmtdev = cdll.LoadLibrary('libmtdev.so.1')
  File "/home/codio/anaconda3/envs/gui/lib/python3.10/ctypes/__init__.py", line 452, in LoadLibrary
(gui) codio@tonightwaiter-sparkurban:~/workspace/app$ python app.py
[INFO   ] [Logger      ] Record log in /home/codio/.kivy/logs/kivy_23-07-25_4.txt
[INFO   ] [Kivy        ] v2.2.1
[INFO   ] [Kivy        ] Installed at "/home/codio/anaconda3/envs/gui/lib/python3.10/site-packages/kivy/__init__.py"
[INFO   ] [Python      ] v3.10.12 | packaged by conda-forge | (main, Jun 23 2023, 22:40:32) [GCC 12.3.0]
[INFO   ] [Python      ] Interpreter at "/home/codio/anaconda3/envs/gui/bin/python"
[INFO   ] [Logger      ] Purge log fired. Processing...
[INFO   ] [Logger      ] Purge finished!
[INFO   ] [Factory     ] 190 symbols loaded
[INFO   ] [Image       ] Providers: img_tex, img_dds, img_sdl2, img_pil (img_ffpyplayer ignored)
[INFO   ] [Text        ] Provider: sdl2
[ERROR  ] [Input       ] MTDev is not supported by your version of linux
Traceback (most recent call last):
  File "/home/codio/anaconda3/envs/gui/lib/python3.10/site-packages/kivy/input/providers/__init__.py", line 41, in <module>
    import kivy.input.providers.mtdev
  File "/home/codio/anaconda3/envs/gui/lib/python3.10/site-packages/kivy/input/providers/mtdev.py", line 94, in <module>
    from kivy.lib.mtdev import Device, \
```

Guide ×

Collapse

Finishing the Application

Styling the Application

Activate the `gui` virtual environment.

```
conda activate gui
```

Let's address the bland appearance of the application. First, we want to better position the widgets. They currently fill the window from left to right and top to bottom. We want to leave some padding around the widgets. Set the `size_hint` to 80% (0.8) of the window width and 95% (0.95) of the window height. That means there will be a little more space to the left and right of the widgets than the space above and below them. Then center the widgets both horizontally and vertically so that the empty space is equally distributed around all the widgets. Use a dictionary to place the center of the widgets 50% (0.5) of the width of the window. In addition, place the center of the widgets 50% (0.5) of the height of the window.

```
    # define the window
    Window.clearcolor = '#34495e'
    Window.size = (150, 300)
    self.window = GridLayout(cols=1)
    self.window.size_hint = (0.8, 0.95)
    self.window.pos_hint = {'center_x' : 0.5, 'center_y' : 0.5}
```

Use the following command to run your application. Click the link to see the output. The widgets should have padding around them on all sides.

```
python app.py
```

Preview the GUI

For the label widgets, we want to increase the font size, make the text bold, and color the text a light gray. **Note**, making the font size for the title 24 is a little too big given the size of the application. You can adjust the size to what you think looks best.

```
    # define the widgets
    title = Label(text='Click a Button',
                  font_size = 18,
                  bold = True,
                  color = '#ecf0f1')
    self.count = 0
    self.count_label = Label(text=str(self.count),
                             font_size = 24,
                             bold = True,
                             color = '#ecf0f1')
```

Next, the buttons will also have an increased font size and bold text. However, the colors will be different. The add button will be green, while the subtract button will be red. Kivy automatically darkens the background colors for buttons, which we do not want. Setting `background_normal` to an empty string does not alter the color.

```
    self.add_button = Button(text='+',
                             font_size = 24,
                             bold = True,
                             background_normal = '',
                             background_color = '#27ae60')
    self.subtract_button = Button(text='-',
                                  font_size = 24,
                                  bold = True,
                                  background_normal = '',
                                  background_color = '#c0392b')
```

Run the program and click the link to see the output. The application should be styled now. However, the buttons do not do anything when clicked.

```
python app.py
```

Preview the GUI

Try this variation:

- Adjust the colors of the application. Search for sites that have hexadecimal colors (like this one). Or, if you want colors that work well together, you can use sites (like this) that provide color palettes.

```
python app.py
```

Preview the GUI

Adding Callbacks

The final thing we need to address is connecting the buttons to the state of the program. We want the number to increase when we click the green button and decrease when we click the red button. To do this, we need to use a callback. This is a method that is attached to a button. More specifically, we can attach a callback to a particular activity like when a button is pressed or is released. Use the `bind()` method to attach a callback to the pressing of each button. The callbacks should go at the end of the `build()` method but before the return statement.

```
    self.add_button.bind(on_press=self.add_callback)
    self.subtract_button.bind(on_press=self.subtract_callback)

    return self.window
```

Outside the `build()` method and still inside the `ClickButtons` class, define the callbacks. Each one should take `self` and `instance` as arguments. For the addition callback, increase `self.count` by one and update the `text` attribute of `self.count_label` with the new value. For the subtraction callback, decrease `self.count` by one and update the label text. Remember, label text must be a string, so typecast `self.count`.

```
  def add_callback(self, instance):
    self.count += 1
    self.count_label.text = str(self.count)

  def subtract_callback(self, instance):
    self.count -= 1
    self.count_label.text = str(self.count)
```

The GUI application should be finished. Execute the command below and click the link to see the output. Click the buttons to make sure the counter properly responds to the clicks.

```
python app.py
```

Preview the GUI

Lastly, deactivate the `gui` virtual environment.

```
conda deactivate
```

▼ **Code**

Your code should look like this:

```
from kivy.app import App
from kivy.uix.gridlayout import GridLayout
from kivy.uix.label import Label
from kivy.uix.button import Button
from kivy.core.window import Window

class ClickButtons(App):
  def build(self):
    # define the window
    Window.clearcolor = '#34495e'
    Window.size = (150, 300)
    self.window = GridLayout(cols=1)
    self.window.size_hint = (0.8, 0.95)
    self.window.pos_hint = {'center_x' : 0.5, 'center_y' : 0.5}

    # define the widgets
    title = Label(text='Click a Button',
                  font_size = 18,
                  bold = True,
                  color = '#ecf0f1')
    self.count = 0
    self.count_label = Label(text=str(self.count),
                             font_size = 24,
                             bold = True,
                             color = '#ecf0f1')
    self.add_button = Button(text='+',
                             font_size = 24,
                             bold = True,
                             background_normal = '',
                             background_color = '#27ae60')
    self.subtract_button = Button(text='-',
                                  font_size = 24,
                                  bold = True,
                                  background_normal = '',
                                  background_color = '#c0392b')

    # add widgets
    self.window.add_widget(title)
    self.window.add_widget(self.add_button)
    self.window.add_widget(self.count_label)
    self.window.add_widget(self.subtract_button)

    self.add_button.bind(on_press=self.add_callback)
    self.subtract_button.bind(on_press=self.subtract_callback)

    return self.window

  def add_callback(self, instance):
    self.count += 1
    self.count_label.text = str(self.count)

  def subtract_callback(self, instance):
    self.count -= 1
    self.count_label.text = str(self.count)

if __name__ == "__main__":
  ClickButtons().run()
```

Reading Question

Drag the blocks into the correct order for building a Kivy application (use our example as a model).

- Inherit from the `App` class
- Define the `build` method
- Define the layout
- Create a widget
- Add a widget to the layout
- Return the layout

Here is the solution:

- Inherit from the `App` class
- Define the `build` method
- Define the layout
- Create a widget
- Add a widget to the layout
- Return the layout

Start your Kivy application by inheriting from the `App` class. Then create the `build` method. Then create the layout, which is needed for any widgets. You cannot add a widget without having first created it. Finally, return the layout.

Check It!

Next